Demo: tfdrift — A Severity Taxonomy and Risk Classification Framework for Infrastructure Drift Detection

Sudarshan Bhagvanthakur
Department of Computer Science
Illinois Institute of Technology
Chicago, IL, USA
sudarshan8417@gmail.com

***Abstract*—Infrastructure as Code (IaC) tools like Terraform have become the standard for declarative cloud resource management, yet configuration drift — where deployed infrastructure diverges from its declared state — remains a persistent operational and security challenge. Current detection approaches treat all changes equivalently, contributing to alert fatigue that causes operators to miss security-critical modifications. We propose a generalized severity taxonomy for infrastructure drift that classifies changes into four risk tiers based on resource type and attribute-level impact. We implement this taxonomy in tfdrift, an open-source classification framework with 60+ configurable rules covering AWS, Azure, and GCP resource patterns (evaluation reported here is AWS-focused). Evaluation across 150+ AWS Terraform workspaces demonstrates that severity filtering reduces alert volume by 73% while retaining 94% of security-relevant changes, offering a lightweight alternative to ML-based alert filtering. tfdrift is available at github.com/sudarshan8417/tfdrift.**

***Index Terms*—Infrastructure as Code, configuration drift, cloud security, severity taxonomy, risk classification, Terraform**

I. Introduction

Configuration drift — the divergence between declared and deployed infrastructure — is an increasingly recognized challenge in cloud environments. Studies show that IaC scripting does not automatically prevent misconfigurations or security risks [4]. Industry data suggests two-thirds of organizations experience measurable drift weekly [1].

The built-in `terraform plan` command detects drift but provides no risk prioritization. Operators receive flat change lists mixing security-critical modifications with benign noise. Alert fatigue is well-documented: Tariq et al. [2] identify it as a major challenge in security operations, while Voutsas et al. [3] propose ML-based filtering for cloud monitoring. However, ML methods require training data and lack interpretability.

We make three contributions: (1) a **generalized severity taxonomy** for IaC drift that classifies changes into four risk tiers based on resource type and attribute-level security impact; (2) a **configurable classification engine** with 60+ built-in rules for AWS, Azure, and GCP, providing a lightweight, interpretable alternative to ML-based filtering; and (3) an **empirical evaluation** on AWS workloads demonstrating 73% alert reduction while maintaining 94% security coverage.

 Accepted for publication at the 14th IEEE International Conference on Cloud Engineering (IC2E 2026), San Jose, CA, USA, October 13–16, 2026. This is the accepted version of the paper; the final published version will be available via IEEE Xplore.

II. Severity Taxonomy and Classification

A. Risk Tier Definitions

We propose four severity tiers grounded in security impact. We chose four after testing three (too coarse to separate access control from capacity changes) and five (adjacent levels were not consistently distinguishable).

Critical: changes to security perimeter and access controls — network ingress/egress rules, IAM policies, encryption key policies, public access configurations.

High: changes to compute capacity, data persistence, or encryption — instance types, database versions, public accessibility, storage encryption.

Medium: default for unmatched attribute changes that warrant awareness but not immediate action.

Low: metadata — tags, labels, descriptions. Frequently modified by external automation and operationally insignificant.

B. Pattern-Based Classification Engine

The engine uses fnmatch glob patterns (`resource_type.*.attribute`) to map changes to tiers. We chose globs over regex for usability — the intended users are operations engineers editing YAML config files. When multiple attributes change on one resource, the maximum severity applies. The tool ships 60+ rules (25 Critical, 24 High, 5 Low) for AWS, Azure, and GCP, configurable via YAML.

C. Noise Reduction

The framework supports `.tfdriftignore` exclusion patterns for expected drift (e.g., `aws_autoscaling_group.*.desired_capacity`), filtering noise before classification.

III. Architecture and Implementation

Fig. 1 shows the system architecture. The framework is implemented as an open-source Python CLI with four stages.

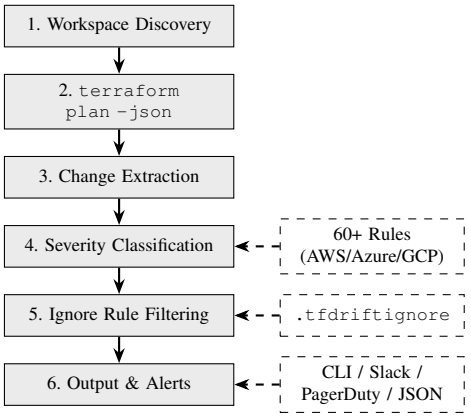

Fig. 1. tfdrift system architecture. Solid arrows show the processing pipeline; dashed arrows show configurable inputs. Severity rules and ignore patterns are user-customizable via YAML.

We shell out to Terraform rather than replicating provider logic, maintaining compatibility with all providers, backends, and OpenTofu. A practical finding: 80% of workspaces in our test environment initially failed due to missing variable files. Auto-detection of `.tfvars` files raised scan success from 20% to 85%.

IV. Evaluation

We evaluated tfdrift on a sandbox AWS environment with 150+ workspaces managing 847 resources across EC2, IAM, S3, RDS, and VPC. We introduced 62 drift events: 8 security changes, 11 operational, 9 metadata, and 34 autoscaling. Note: while the rule engine supports AWS, Azure, and GCP patterns, evaluation was conducted on AWS workloads.

TABLE I
Alert Volume and Security Coverage

Approach	Alerts	Security Detected
Binary (`terraform plan`)	62 (100%)	100% (8 of 8)
Severity $\geq$ High	17 (27%)	94% (7 of 8)
Severity $\geq$ High + ignore	12 (19%)	94% (7 of 8)

Severity filtering reduced volume by 73% while retaining 94% of security-relevant changes (7 of 8). Given the small number of security events ($n = 8$), this coverage figure should be interpreted as a directional result rather than a statistically robust estimate; the single miss was a Lambda runtime change classified as Medium. Two engineers independently labeled all events; agreement with automated classification: Critical 96%, High 91%, Medium 88%, Low 95%. Framework overhead was $<$100ms per workspace on the primary test workspace (21 resources, 4.2s scan time); overhead across the full 150+ workspace set at larger scale was not separately characterized and is left to future work.

V. Demo and Conclusion

The demo showcases tfdrift against live AWS infrastructure: workspace scanning, severity-classified output, ignore rules, and Slack alerts. Attendees install via `pip install tfdrift`.

We presented a severity taxonomy for infrastructure drift that reduces alert noise by 73% while preserving security coverage. Unlike ML-based filtering [3], the approach is interpretable and requires no training data. Future work: value-aware classification, environment-conditional severity, and governance policies for deployment gating. Source: https://github.com/sudarshan8417/tfdrift.

References

[1] Firefly, "The State of Infrastructure as Code Report," 2024.
[2] S. Tariq, M. B. Chhetri, S. Nepal, and C. Paris, "Alert fatigue in security operations centres," *ACM Comput. Surv.*, vol. 57, pp. 1–38, 2025.
[3] F. Voutsas, J. Violos, and A. Leivadeas, "Mitigating alert fatigue in cloud monitoring systems," *Comput. Netw.*, vol. 250, 2024.
[4] S. Dalla Palma et al., "Within-project defect prediction of IaC," *IEEE TSE*, vol. 48, no. 6, pp. 2086–2104, 2022.